

21世纪软件工程专业规划教材

软件工程导论（第6版）

第5章 详细设计

- **根本目标**：确定应该怎样具体地实现所要求的系统，经过这个阶段的设计工作，应该得出对目标系统的精确描述。
- 详细设计阶段的任务不是具体地编写程序，而是要设计出程序的“蓝图”
详细设计的结果基本上决定了最终的程序代码的质量。
- 详细设计的目标不仅是逻辑上正确地实现每个模块的功能，更重要的是设计出的处理过程应该尽可能简明易懂。

目录

CONTENTS

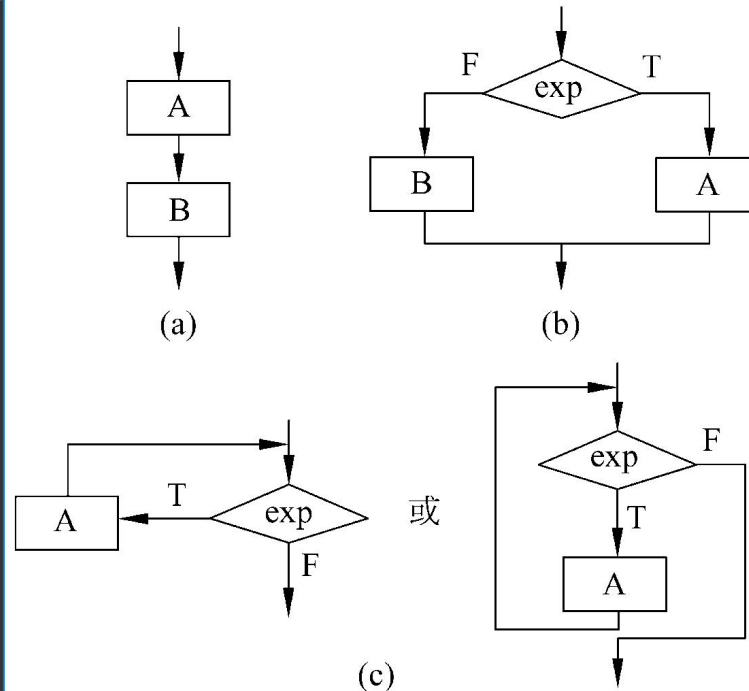
- 1 结构程序设计
- 2 人机界面设计
- 3 过程设计的工具
- 4 面向数据结构的设计方法
- 5 程序复杂程度的定量度量

结构程序设计

结构程序设计的概念最早由E.W.Dijkstra(艾兹格·W·迪科斯彻)提出。他是计算机先驱之一，开发了程序设计的框架结构。



- 1965年，E.W.Dijkstra提出**程序的质量与程序中所包含的GO TO 语句的数量成反比**
- 1966年Bohm和Jacopini证明了只用“**顺序**”、“**选择**”和“**循环**”控制结构就能实现任何单入口单出口的程序。



- 1968年Dijkstra再次建议从一切高级语言中**取消GO TO**语句，只使用3种基本控制结构写程序。
学界认识到不是简单地去掉GO TO 语句的问题，而是要创立一种新的程序设计思想、方法和风格。
- 1971年IBM公司在纽约时报信息库管理系统的设计中成功地使用了结构程序设计技术；
- 1972年IBM公司的Mills进一步提出，程序应该**只有一个入口和一个出口**，补充了结构程序设计的规则。

实际上用顺序结构和循环结构（又称DO-WHILE结构）完全可以实现选择结构（又称IF-THEN-ELSE结构），因此，理论上最基本的控制结构只有两种。

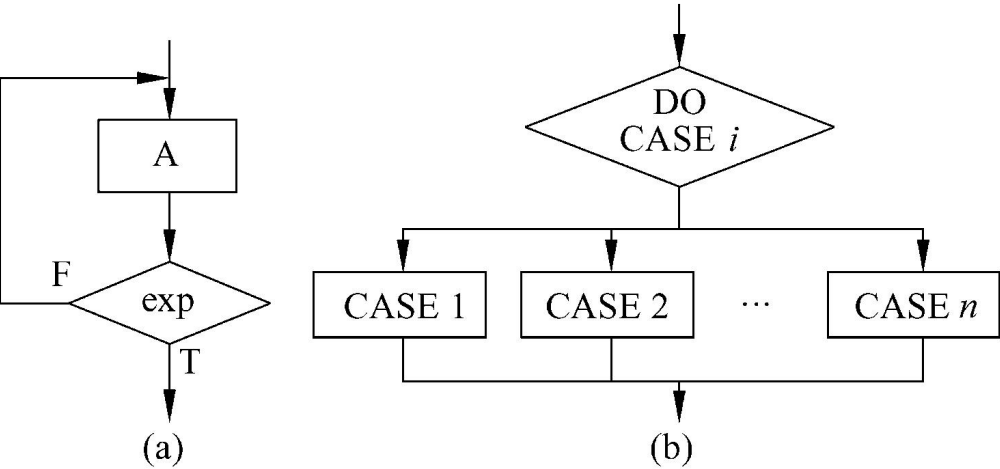
什么是结构程序设计？

结构程序设计**经典定义**：如果一个程序的代码块仅仅通过**顺序**、**选择**和**循环**这3种基本控制结构进行连接，并且每个代码块只有一个**入口**和一个**出口**，则称这个程序是结构化的。

结构程序设计**更全面的定义**：结构程序设计是尽可能**少用GO TO**语句的程序设计方法。最好仅在检测出错误时才使用GO TO语句，而且应该总是使用前向GO TO语句。

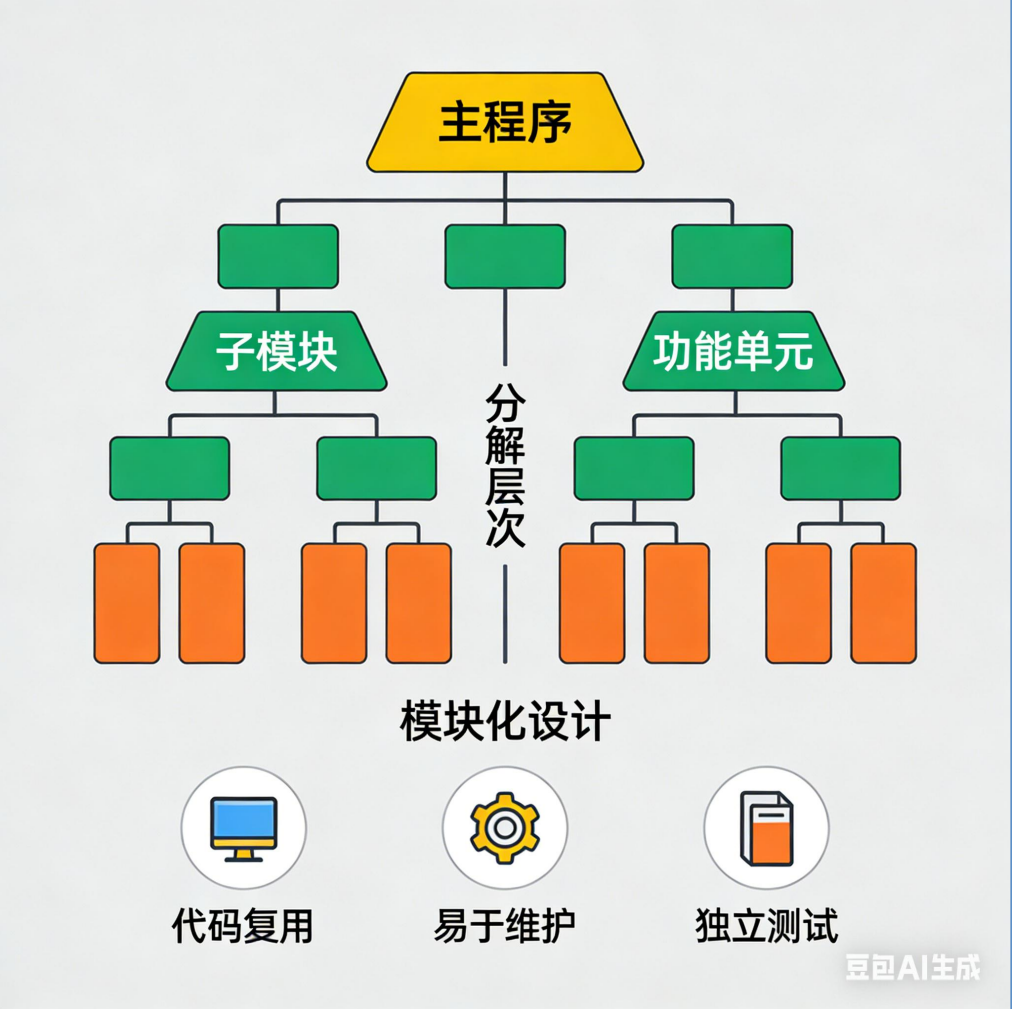


从理论上说只用上述3种基本控制结构就可以实现任何单入口单出口的程序，但是为了实际使用方便起见，常常还允许使用DO-UNTIL和DO-CASE两种控制结构。



- 如果只允许使用顺序、IF-THEN-ELSE型分支和DO-WHILE型循环这3种基本控制结构，则称为经典的结构程序设计；
- 如果除了上述3种基本控制结构之外，还允许使用DO-CASE型多分支结构和DO-UNTIL型循环结构，则称为扩展的结构程序设计；
- 如果再允许使用LEAVE（或BREAK）结构，则称为修正的结构程序设计。

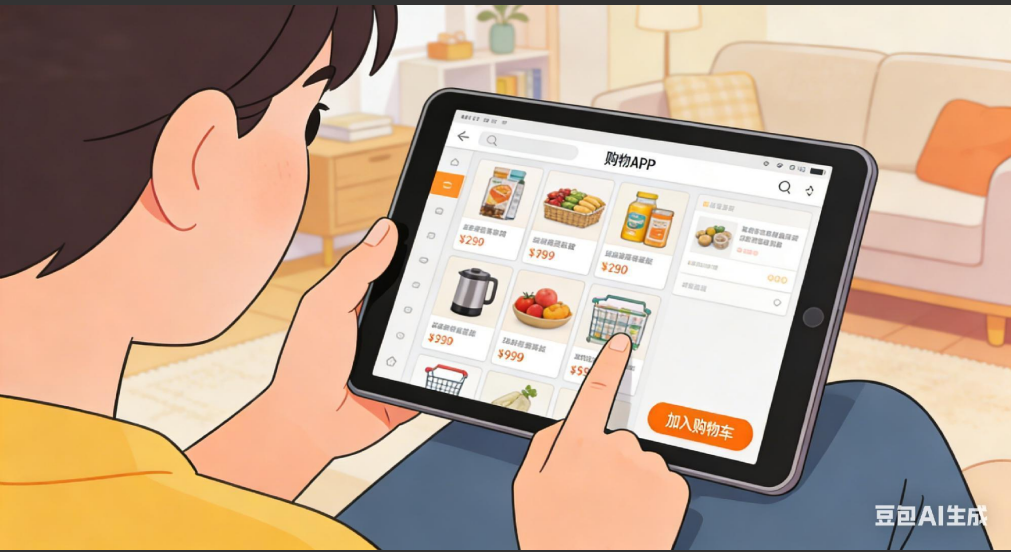
“面向结构”的程序设计方法即结构化程序设计方法，是“面向过程”方法的改进，结构上将软件系统划分为若干功能模块，各模块按要求单独编程，再由各模块连接，组合构成相应的软件系统。该方法强调程序的结构性，所以容易做到易读，易懂。该方法思路清晰，做法规范，深受设计者青睐。



人机界面设计

人机界面设计 (Human-Machine Interface Design, HMI)

是人与机器之间**信息交互的媒介设计**，旨在通过优化界面布局、交互逻辑和视觉表现，实现高效、舒适的人机协作。它融合了计算机科学、认知心理学、人机工程学等多学科知识，核心目标是提升系统的**易用性和用户体验**。



人机界面设计是用户与机器（如计算机、工业设备等）**传递指令和反馈信息的桥梁**，涵盖**硬件**（如触摸屏、按钮）和**软件**（如图形界面、操作逻辑）的设计。

场景	典型操作	建议目标响应时间	用户容忍度
普通网页浏览	点击链接、打开页面	< 2秒 (首屏)	较低，超3秒大量流失
Web后台/内部系统	查询订单、保存配置	< 1秒 (列表/表单提交)	中等，员工会抱怨但被迫使用
移动App	滑动加载、下拉刷新	< 300毫秒 (交互反馈)	很低，卡顿立即差评
API接口	获取数据、同步信息	< 100毫秒 (P99)	视调用方而定，上游系统可能超时熔断
实时系统	股票交易、游戏操作	< 10毫秒	极高，几毫秒差异影响收益/成败
批处理任务	生成报表、导出数据	数秒~数分钟	高，用户可接受等待，但需进度提示

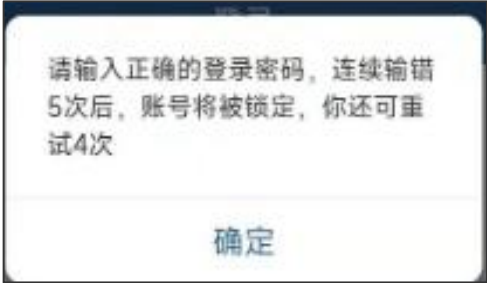
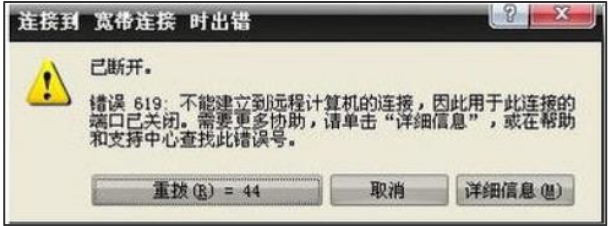
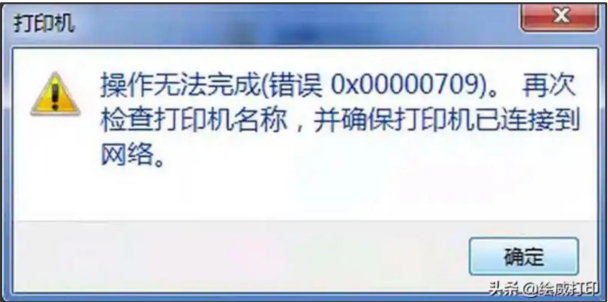
② **用户帮助设施**：用户在遇到复杂问题时需要查看用户手册以寻求答案，大多数现代软件都提供**联机帮助设施**，用户无需离开用户界面就能解决自己的问题。



- ① 在用户与系统交互期间，是否在任何时候都能**获得**关于系统任何功能的帮助信息？
- ② 用户怎样**请求帮助**？（帮助菜单，特殊功能键和HELP命令）
- ③ 怎样**显示帮助**信息？在独立的窗口, 在屏幕固定位置显示简短提示)
- ④ 用户怎样**返回**到正常的交互方式中？（返回按钮, 功能键）
- ⑤ 怎样**组织帮助**信息？（平面结构，信息的层次结构, 超文本结构）

③ **出错信息处理：** 出错信息和警告信息，是出现问题时交互式系统给出的“坏消息”。一般说来，交互式系统给出的出错信息或警告信息，具有下述属性：

- ① 用用户**可以理解**的术语描述问题。
- ② 提供有助于从错误中恢复的**建设性意见**。
- ③ 指出错误可能导致哪些**负面后果**（例如，破坏数据文件）以使用户检查是否出现了这些问题，并在确实出现问题时及时解决。
- ④ 伴随着听觉上或视觉上的**提示**
- ⑤ 不能带有**指责色彩**，不能责怪用户。



④ **命令交互：** 许多高级用户仍然偏爱面向命令行的交互方式。在提供命令交互方式时，必须考虑下列设计问题。

PPT 常用快捷键	
新建演示文稿	Ctrl+N
使所选文字加粗	Ctrl+B
更改所选文本的字体大小	Alt + H、 F、 S
打开了缩放对话框	Alt+W, Q
剪切所选的文本、 对象或幻灯片	Ctrl+X
复制所选的文本、 对象或幻灯片	Ctrl+C
粘贴剪切或复制的文本、 对象或幻灯片	Ctrl+V
撤消上一步操作	Ctrl+Z
保存演示文稿	Ctrl+S
插入图片	Alt+N, P
插入形状	Alt + H、 S、 H



- ① 是否每个菜单选项都有对应的命令？
- ② 采用何种命令形式？（控制序列，例如，Ctrl+P），功能键和输入命令。
- ③ 学习和记忆命令的难度有多大？忘记了命令怎么办？
- ④ 用户是否可以定制或缩写命令？

在越来越多的应用软件中，人机界面设计者都提供了“**命令宏机制**”。（命令宏的本质是“**记录操作 → 生成代码 → 重复执行**”，适用于：重复性任务自动化，自定义功能扩展，提高效率。掌握宏的使用可以极大提升办公效率！

过程设计的工具

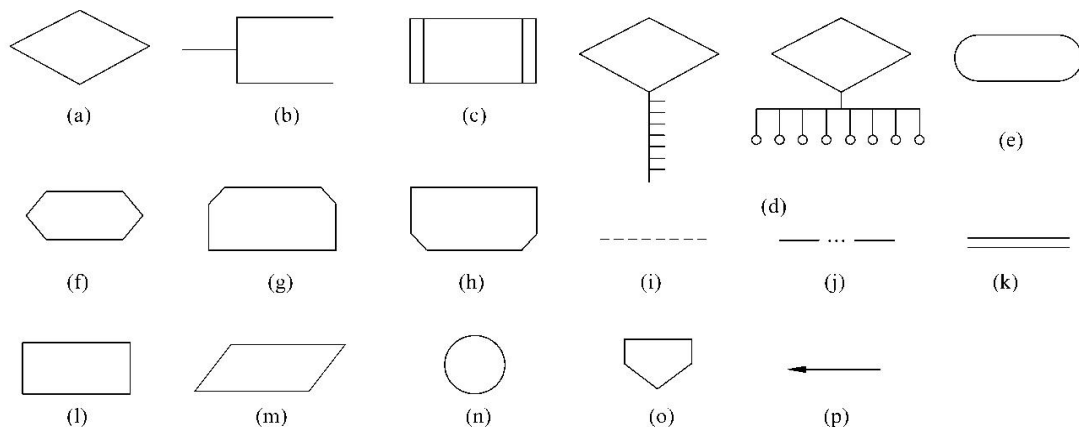
过程设计的工具：描述程序处理过程的工具称为过程设计的工具。它们可以分为**图形**、**表格**和**语言**三类。

它们可以分为**图形**、**表格**和**语言**三类。

无论是哪种工具，对它们的基本要求都是能提供对设计的**无歧义的描述**，能够指明控制流程、处理功能、数据组织及其他方面的实现细节，**从而在编码阶段能把对设计的描述直接翻译成程序代码。**

设计的描述直接翻译成程序代码。

又称为程序框图，它是使用最广泛的描述过程设计的方法。

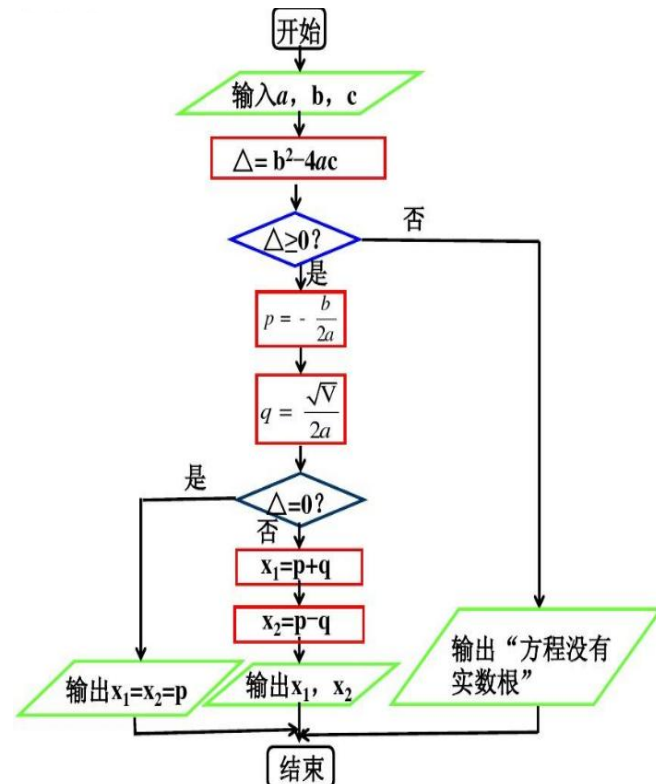


优势：

- ① 直观易懂：比纯文字描述更清晰。
- ② 逻辑清晰：有助于梳理复杂逻辑。
- ③ 语言无关：是设计和沟通阶段的通用工具。

局限性：

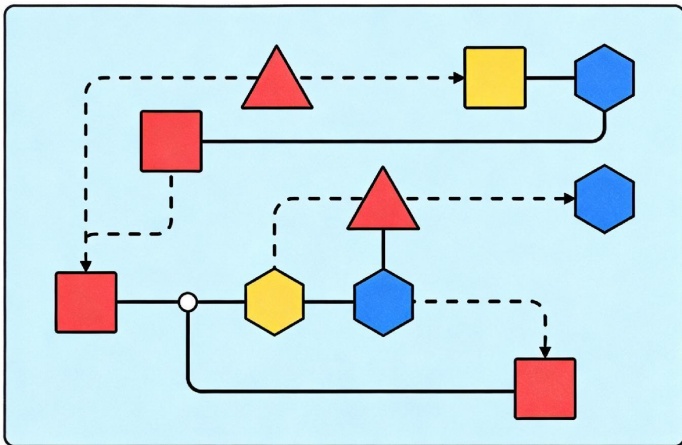
- ① 难以描述细节：不适合描述非常复杂或数据密集型算法的细节。
- ② 维护困难：当程序频繁修改时，同步更新流程图可能很麻烦。
- ③ 不适用于面向对象设计：对于类、对象、消息传递等，UML图（如活动图、序列图）更合适。



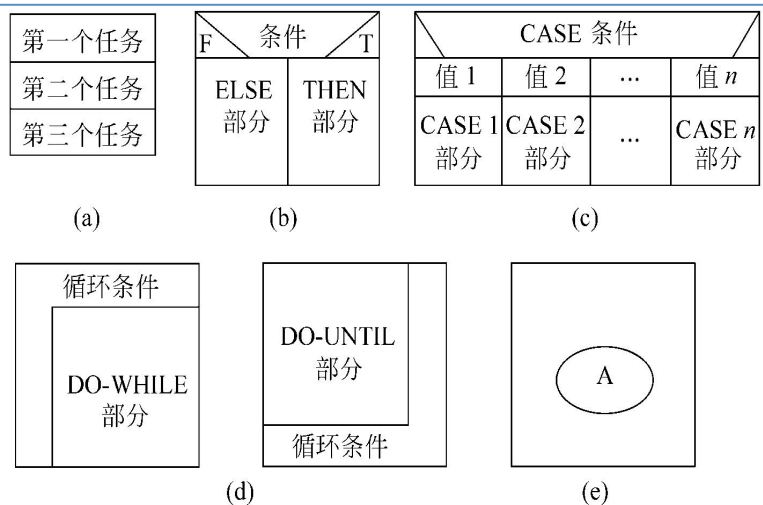
过程设计工具

属性

- 大选流程
- 颜色
- 文本内容



出于要有一种不允许违背结构程序设计精神的图形工具的考虑，Nassi和Shneiderman提出了盒图，又称为**N-S图**。

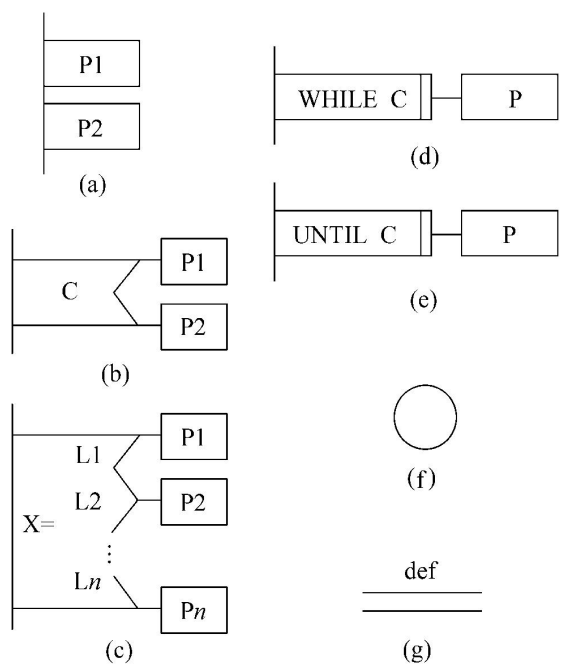


- a. 顺序结构;
- b. IF_THEN_ELSE型;
- c. CASE型;
- d. 循环结构;
- e. 调用子程序A



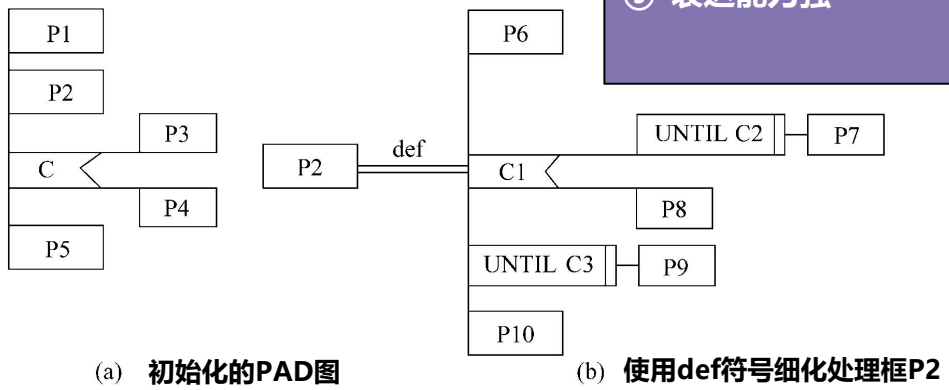
- ① 无流程线
- ② 结构化
- ③ 不可能任意转移控制
- ④ 功能域明确
- ⑤ 很容易确定局部和全程数据的作用域。
- ⑥ 很容易表现嵌套关系，也可以表示模块的层次结构。

问题分析图（problem analysis diagram）的英文缩写，用**二维树形结构**的图来表示程序的控制流。



- a. 顺序（先执行P1后执行P2）；
- b. 选择（IF C THEN P1 ELSE P2）；
- c. CASE型多分支；
- d. WHILE型循环（WHILE C DO P）；
- e. UNTIL型循环（REPEAT P UNTIL C）；
- f. 语句标号；
- g. 定义

- ① 优秀的结构化支持
- ② 强大的设计支持
- ③ 极佳的代码生成性
- ④ 易于理解和维护
- ⑤ 表达能力强

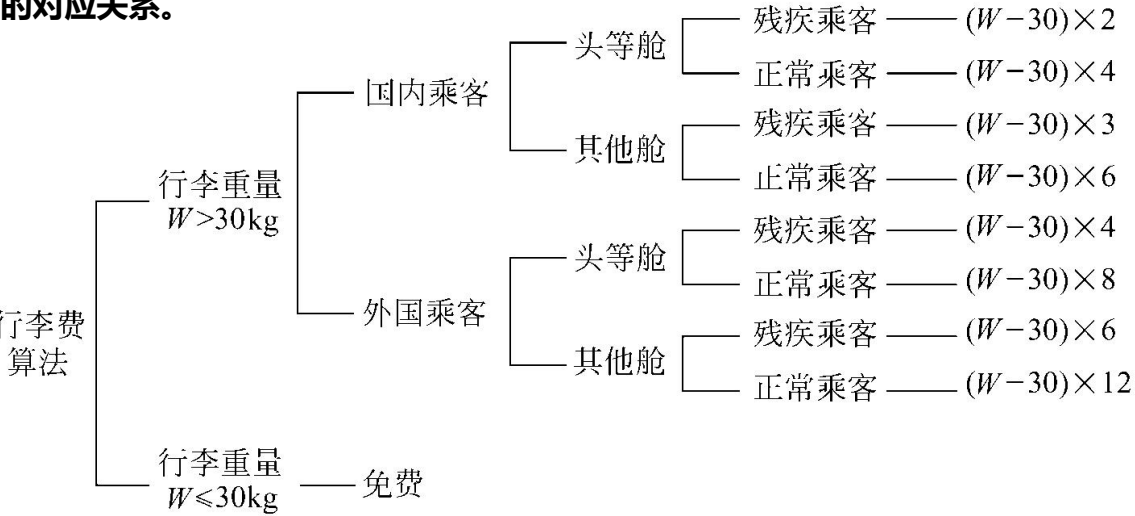


判定表能够清晰地表示复杂的条件组合与应做的动作之间的对应关系。判定表由4部分组成：

- **左上部**：列出所有条件
- **左下部**：是所有可能做的动作
- **右上部**：是表示各种条件组合的一个矩阵
- **右下部**：是和每种条件组合相对应的动作。

	1	2	3	4	5	6	7	8	9
国内乘客		T	T	T	T	F	F	F	F
头等舱		T	F	T	F	T	F	T	F
残疾乘客		F	F	T	T	F	F	T	T
行李重量 $W \leq 30\text{kg}$	T	F	F	F	F	F	F	F	F
免费	X								
$(W-30) \times 2$				X					
$(W-30) \times 3$					X				
$(W-30) \times 4$		X						X	
$(W-30) \times 6$			X						X
$(W-30) \times 8$						X			
$(W-30) \times 12$							X		

判定树是判定表的变种，它也能清晰地表示复杂的条件组合与应做的动作之间的对应关系。



④ **过程设计语言：过程设计语言（PDL）**也称为伪码。是用**正文形式**表示数据和过程的设计工具。

```
输入：训练集  $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ;  
属性集  $A = \{a_1, a_2, \dots, a_d\}$ .  
过程：函数 TreeGenerate( $D, A$ )  
1: 生成结点 node;  
2: if  $D$  中样本全属于同一类别  $C$  then  
3:   将 node 标记为  $C$  类叶结点; return  
4: end if  
5: if  $A = \emptyset$  OR  $D$  中样本在  $A$  上取值相同 then  
6:   将 node 标记为叶结点, 其类别标记为  $D$  中样本数最多的类; return  
7: end if  
8: 从  $A$  中选择最优划分属性  $a_*$ ;  
9: for  $a_*$  的每一个值  $a_*^v$  do  
10: 为 node 生成一个分支; 令  $D_v$  表示  $D$  中在  $a_*$  上取值为  $a_*^v$  的样本子集;  
11: if  $D_v$  为空 then  
12:   将分支结点标记为叶结点, 其类别标记为  $D$  中样本最多的类; return  
13: else  
14:   以 TreeGenerate( $D_v, A \setminus \{a_*\}$ ) 为分支结点  
15:   end if  
16: end for  
输出：以 node 为根结点的一棵决策树
```

- PDL**有下述优点：
- 可以作为注释直接插在源程序中间。
 - 可以使用普通的正文编辑程序或文字处理系统，很方便地完成**PDL**的书写和编辑工作。
 - 已经有**自动处理PDL的程序**存在，而且可以自动由**PDL**生成程序代码。**PDL**的缺点是不如图形工具形象直观，描述复杂的条件组合与动作间的对应关系时，不如判定表清晰简单

图 4.2 决策树学习基本算法

面向数据结构的设计方法

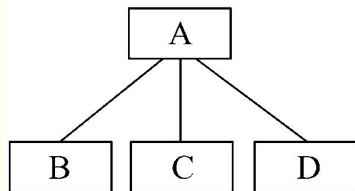
面向数据结构的设计方法的最终目标是得出**对程序处理过程的描述**，因此最适用于详细设计阶段。也就是说，在完成了软件结构设计之后，可以使用面向数据结构的方法来设计**每个模块**的处理过程。

程序的模块划分、控制流、调用关系，都应该与输入/输出数据的结构相对应。是数据在驱动设计！

Jackson方法

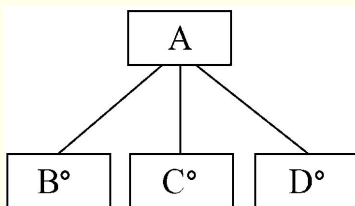
Warnier方法

Jackson 图



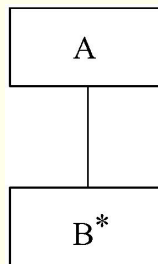
顺序结构

数据由一个或多个数据元素组成，每个元素按确定次序出现一次。**A**由三个元素顺序组成，出现次序：**B C D**



选择结构

数据包含两个或多个数据元素，每次使用这个数据时按照一定条件从这些元素数据中选择一个。**A**是**B或C或D**中的某一个

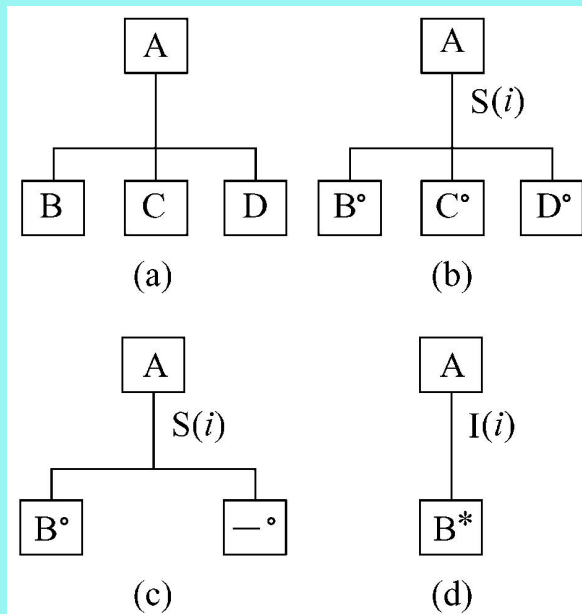


重复结构

数据根据使用时的条件由一个数据元素出现零次或多次构成。**A**由**B**出现**N**次 ($N \geq 0$) 组成

改进的Jackson 图

- ① 顺序结构，B、C、D中任一个都不能是选择出现或重复出现的数据元素（即不能是右上角有小圆圈或星号标记的元素）；
- ② 选择结构，S右面括号中的**数字i**是分支条件的编号；
- ③ **可选结构**，A或者是元素B或者不出现；
- ④ 重复结构，**循环结束条件的编号为i**。

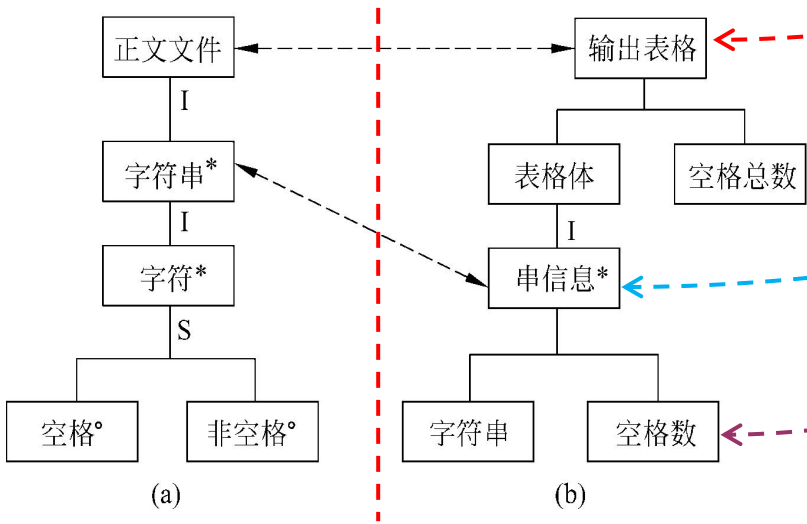


Jackson结构程序设计方法基本上由下述5个步骤组成。下面通过一个例子来进行说明：

例：一个正文文件由若干个**记录**组成，每个记录是一个**字符串**。要求统计每个记录中**空格字符**的个数，以及文件中**空格字符的总个数**。要求的输出数据格式是一个表格，每复制一行输入字符串之后，另起一行印出这个字符串中的空格数，最后印出文件中空格的总个数。

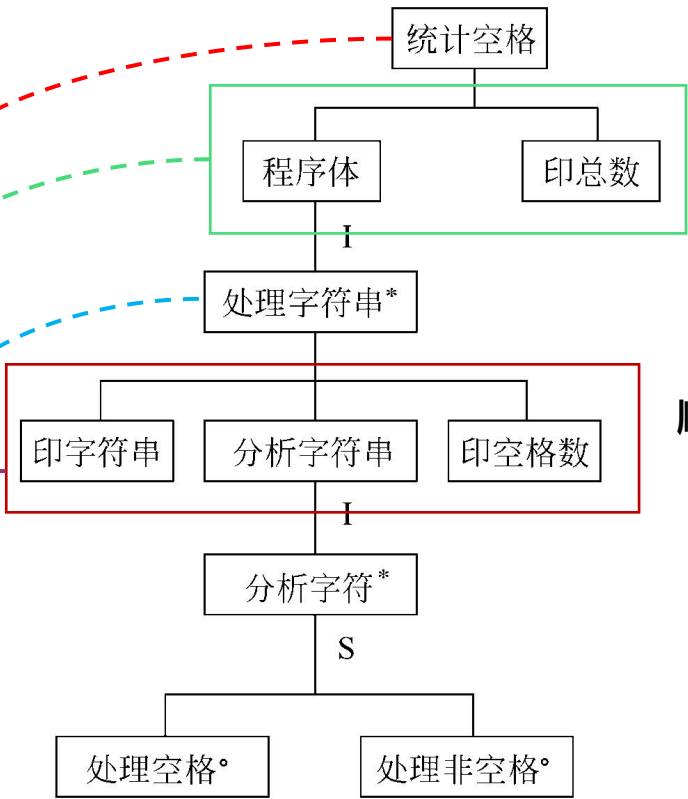
第一步：分析并确定输入数据和输出数据的逻辑结构，并用Jackson图描绘这些数据结构。

第二部：找出**输入**数据结构和**输出**数据结构中有**对应关系**的数据单元。



第三步：用下述3条规则从描绘数据结构的Jackson图导出描绘程序结构的Jackson图。

- ① 为每对有**对应关系的数据单元**，按照它们在数据结构图中的**层次**在程序结构图的相应层次画一个处理框。
- ② 根据输入数据结构中剩余的每个数据单元所处的层次，在程序结构图的相应层次分别为它们画上对应的处理框。



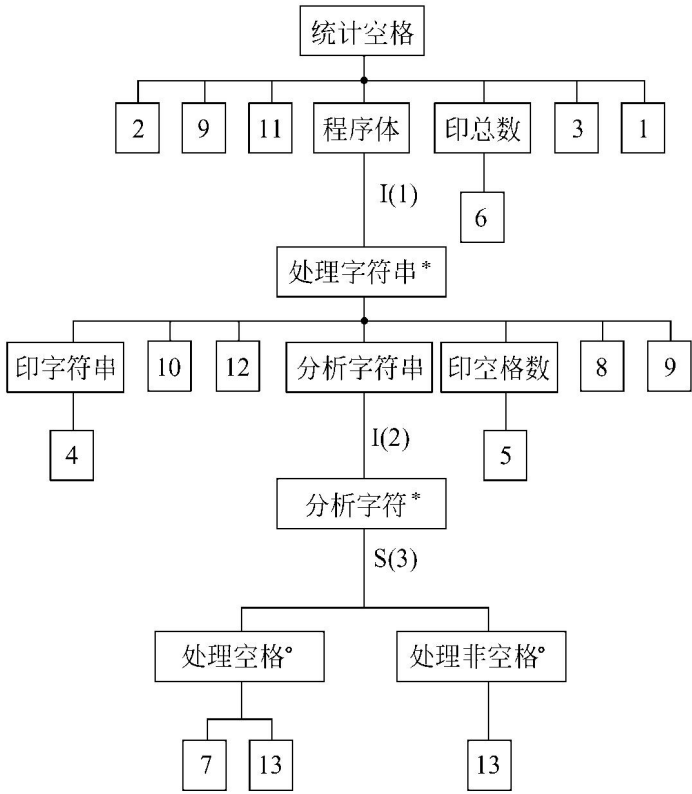
顺序执行，不允许有选择或重复执行的处理

第四步：列出所有**操作和条件**（包括分支条件和循环结束条件），并且把它们分配到程序结构图的适当位置。

统计空格个数需要的全部操作和条件如下：

- (1) 停止
- (2) 打开文件
- (3) 关闭文件
- (4) 印出字符串
- (5) 印出空格数目
- (6) 印出空格总数
- (7) $sum := sum + 1$**
- (8) $totalsum := totalsum + sum$**
- (9) 读入字符串
- (10) $sum := 0$
- (11) $totalsum := 0$
- (12) $pointer := 1$
- (13) $pointer := pointer + 1$**
- I(1) 文件结束
- I(2) 字符串结束
- S(3) 字符是空格

经过简单分析不难把这些操作和条件分配到程序结构图的适当位置：



第五步：用伪码表示程序处理过程。

统计空格seq

打开文件

读入字符串totalsum := 0

程序体iter until文件结束

处理字符串seq

印字符串seq

印出字符串

印字符串end

sum := 0

pointer := 1

分析字符串iter until字符串结束

分析字符select字符是空格

处理空格Seq

sum := sum + 1

pointer := pointer + 1

处理空格end

分析字符or字符不是空格

处理非空格seq

pointer := pointer + 1

处理非空格end

分析字符end

分析字符串end

印空格数seq

印出空格数目

印空格数end

totalsum := totalsum + sum

读入字符串

处理字符串end

程序体end

印总数seq

印出空格总数

印总数end

关闭文件

停止

统计空格end

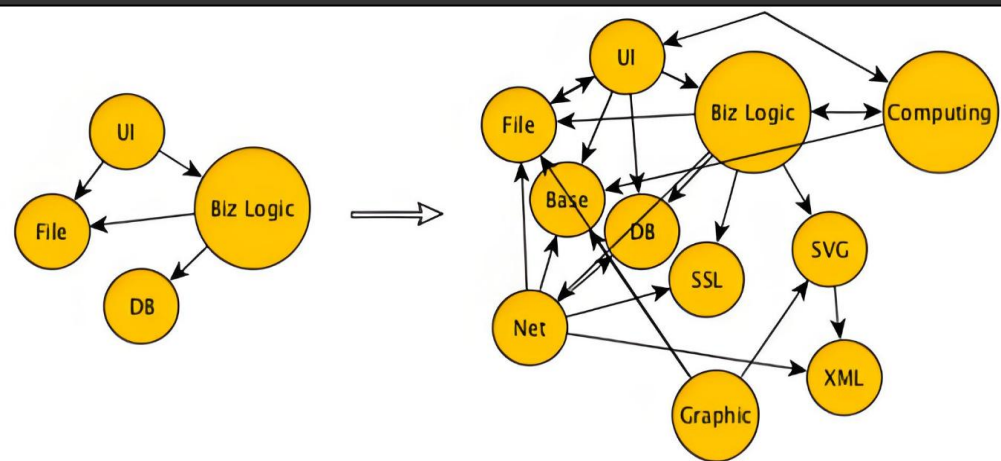
程序复杂程度的定量度量

定量度量程序复杂程度的方法很有价值：

(1) 把程序的复杂程度乘以适当常数即可估算出软件中错误的数量以及软件开发需要用的工作量；

(2) 定量度量的结果可以用来比较两个不同的设计或两个不同算法的优劣；

(3) 程序的定量的复杂程度可作为模块规模的精确限度。



McCabe方法根据程序控制流的复杂程度定量度量程序的复杂程度，这样度量出来的结果称为程序的**环形复杂度**。

三个基本步骤：

- ① 绘制程序流程图
- ② 将程序流程图映射为控制流图
- ③ 计算控制流图的环形复杂度

控制流图实质上是“退化了的”程序流程图，描绘程序的**控制流程**，不表现对数据的具体操作以及分支或循环的具体条件。

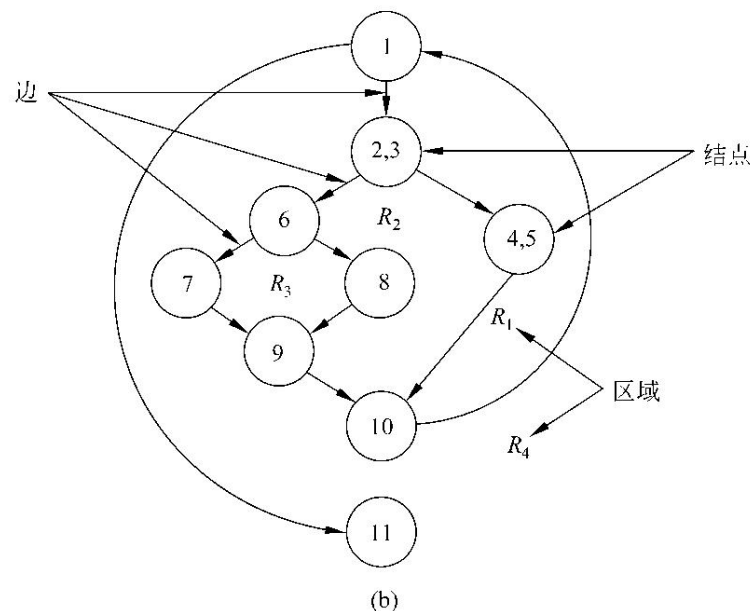
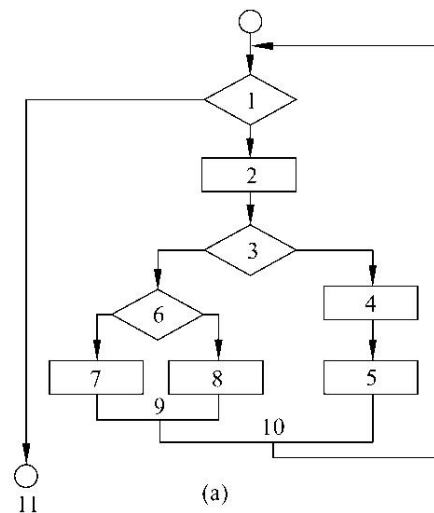
控制流图的特点：

- ① 一个圆代表一条或多条语句；
- ② 一个顺序处理框序列和一个菱形判定框可以映射为一个结点；
- ③ 流图中的箭头线称为边，代表控制流；
- ④ 在流图中一条边必须终止于一个结点。

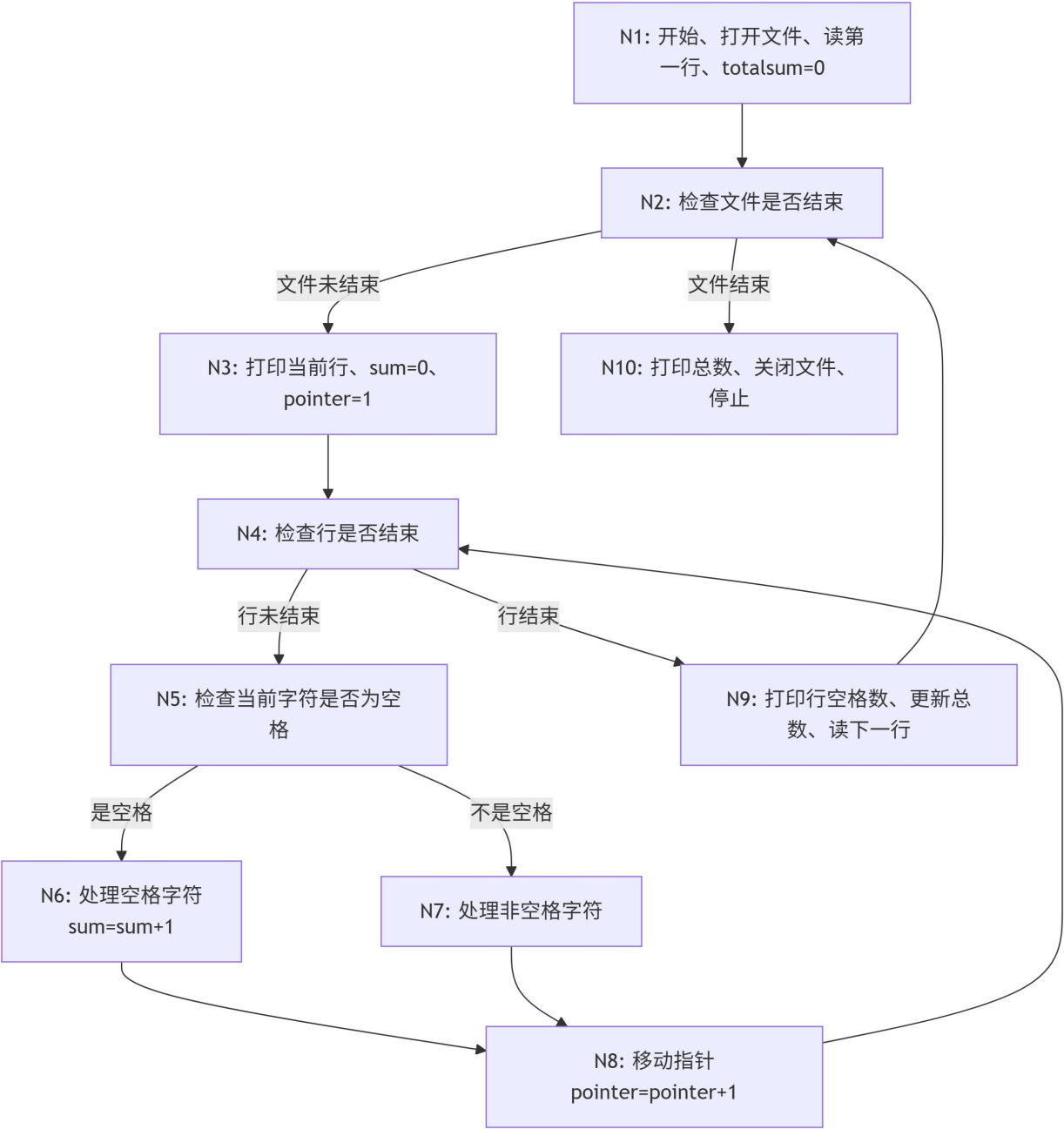
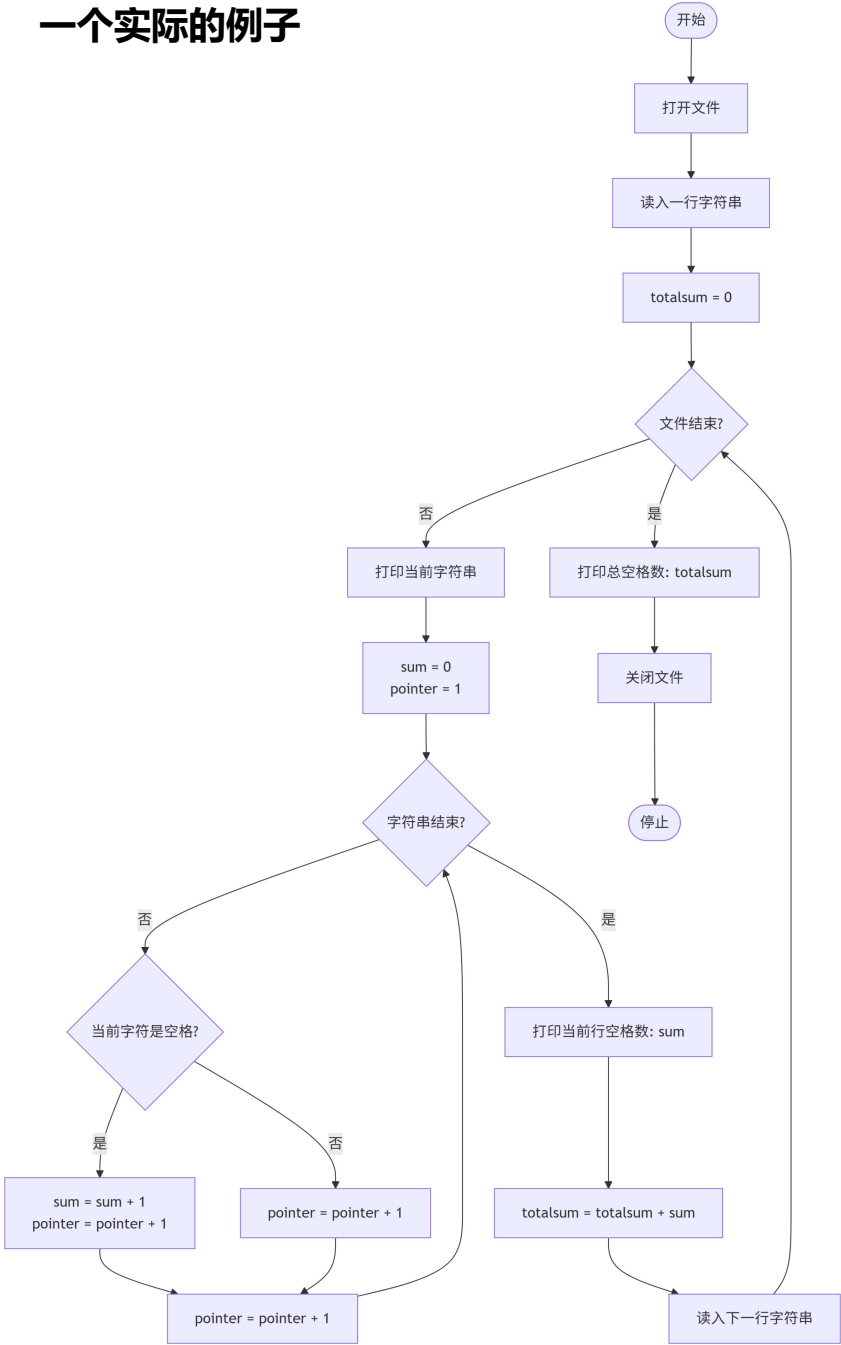
计算环形复杂度的方法

- (1) 流图中线性无关的区域数等于环形复杂度。
- (2) 流图G的环形复杂度 $V(G) = E - N + 2$ ，其中，E是流图中边的条数，N是结点数。
- (3) 流图G的环形复杂度 $V(G) = P + 1$ ，其中，P是流图中判定结点的数目。

上述流图的环形复杂度为4。 $V(G) = 10$ 是上限



一个实际的例子



 **THANKS** 